

INTERNAL TECHNICAL GUIDE

Focused Tools & Package Delivery

How PULSE is delivered as a full application plus focused single-area tools, what each tool is for, and how to publish and verify one.

OPERATING STANDARD

One codebase. One set of records. A focused tool is a narrower door into the same building, never a separate system.

PACKAGES

Eight delivered artifacts

RECORDS

One shared SharePoint schema

SCOPE

Set by PULSE_PORT_CONFIG

PROOF

A write visible in the full app

PACKAGE SET

01 / THE DELIVERED PACKAGE SET

PULSE ships as one application and seven focused packages. Every package is built from the same source tree and reads and writes the same PULSE Lists. A record created in a focused tool is the same record the full application shows; there is no separate store, no synchronisation step, and no reconciliation to perform.

WHAT EACH PACKAGE IS FOR

PACKAGE	SCOPE	TYPICAL PLACEMENT
PULSE	The full application, every area	Team or program landing page
Travel Request Forms	Travel and debrief intake only	Page where people file requests
My Travel	A traveller's own requests and state	Personal or team travel page
Travel Calendar	Team travel and events on one calendar	Planning or scheduling page
Tickets	Support intake and resolution	Help or support page
PULSE Calendar	Standalone team calendar	General team page
PULSE CODE	In-browser editor for the source	Maintainer-only page
PULSE Documentation	This searchable source library	Documentation page

Build every artifact in one pass with the unified builder, which writes each package to both the Firepit package folder and the releases folder and fails the whole run if any package fails.

- `node apps/PULSE/scripts/build-travel-packages.js`

HOW SCOPE IS SET

02 / HOW A FOCUSED TOOL IS BUILT

A focused package is an ordinary PULSE entry file. Before the application loads, the entry file sets a global configuration object that names the mode, the default route, the navigation items, and the sidebar tools. The application reads it during boot and restricts navigation and the sidebar to that area.

Scope is therefore a presentation decision, not a different build of the product. The same modules, the same data layer, and the same permission behaviour are present in every package.

MAINTENANCE RULE

A focused entry file carries the same script set as the full application. Removing scripts to make a package smaller has previously broken boot in ways that only appear once the package is hosted. Change the port configuration, never the script set.

WHAT THIS MEANS IN PRACTICE

- Navigation is limited by design; to reach another area a user opens the full PULSE page.
- Role-aware controls behave identically, so a focused tool does not grant access a user would not otherwise have.
- A fix to a shared module reaches every package on the next build, which is why packages are rebuilt together.

TRAVEL

03 / MY TRAVEL

My Travel presents only the signed-in user's travel records, so a traveller can answer "where does my request stand" without reading the team's list. It is reached in the full application at Travel, then Travel, then My Travel, and is also delivered as its own package.

STATE TABS

TAB	SHOWS
All	Every request belonging to the user
Upcoming	Approved or submitted trips still ahead
Submitted	Awaiting a decision
Withdrawn	Withdrawn by the requester
Cancelled	Cancelled after submission
Completed	Past trips, including any outstanding debrief

Each row reports the user's role on the trip, destination, dates, request status, customer-concurrence state, charge-object state, and whether a debrief is still owed. Row actions cover viewing, editing, withdrawing, and adding the trip to a calendar. Switching to All Travel shows the team's trips where the user's role permits it.

NOTE

An empty Upcoming tab means the signed-in user has no forthcoming trips. It is not evidence that the team has none; use All Travel or the travel calendar for that question.

TRAVEL

04 / TRAVEL CALENDAR AND EVENTS

The travel calendar places team travel and team events on a single calendar so overlapping absences and coverage gaps are visible before a request is approved. It is reached at Travel, then Calendar, and is also delivered as its own package.

Events are the non-travel entries that appear alongside trips and are maintained at Travel, then Events. Trips appear from approved travel requests; they are not entered on the calendar directly.

USE IT BEFORE APPROVING

1. Open the calendar for the requested dates.
2. Check for travellers already away from the same project or function.
3. Check for team events that require the requester to be present.
4. Record the coverage decision with the approval so the reasoning survives.

The calendar is a planning view over existing records. Changing a trip is done on the travel request, and the calendar reflects it after refresh.

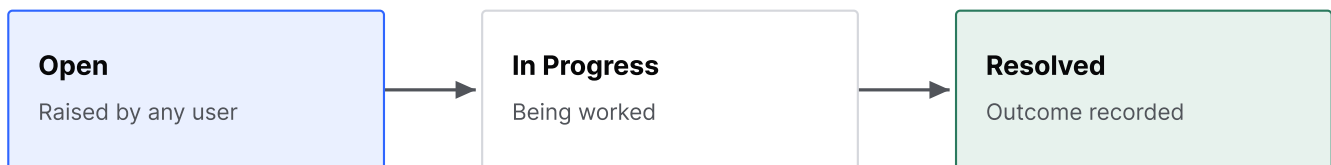
SUPPORT

05 / TICKETS AND ISSUE INTAKE

Support tickets and issue reports are the same records. Both are stored in the PULSE Issues list; there is no separate ticket list in the schema. The repository maps the ticket record kind onto that list and translates the ticket type and status vocabularies onto the issue vocabulary in both directions.

IMPORTANT

A ticket opened in the standalone Tickets package is the same record the full application shows. Do not open a second ticket for the same problem in the other surface.



Source: ticket status values and the repository mapping

STATUS VALUES AS DISPLAYED

TICKET VALUE	MEANING
Open	Recorded, not yet being worked
In Progress	Being worked
Resolved	Closed with an outcome recorded

A useful ticket states the affected route, the project or record identifier, the expected result, the actual result, and the time. Post updates on the ticket record rather than by message, so the history stays complete and the resolution is auditable.

MAINTENANCE

06 / PULSE CODE

PULSE CODE is an in-browser editor used to maintain the PULSE source from a SharePoint page. It is a maintainer tool, not an end-user area, and it is the one package in the set whose behaviour has not been functionally exercised in the documentation workspace.

CAUTION

Only the build of PULSE CODE is verified in this package. Its authentication, file read and write behaviour, and assistant features require confirmation on an approved site before operational use, and its authorisation for use against production source is an open policy question.

BEFORE IT IS USED AGAINST REAL SOURCE

- Confirm which roles are authorised to edit source, and from which pages.
- Confirm the identity mechanism and what it can reach.
- Confirm that edits follow the organisation's change-control and peer-review process rather than bypassing it.
- Confirm data-handling rules for any assistant capability before enabling it.

Treat an edit made in PULSE CODE as a source change subject to the same release validation as any other, including rebuilding the affected packages.

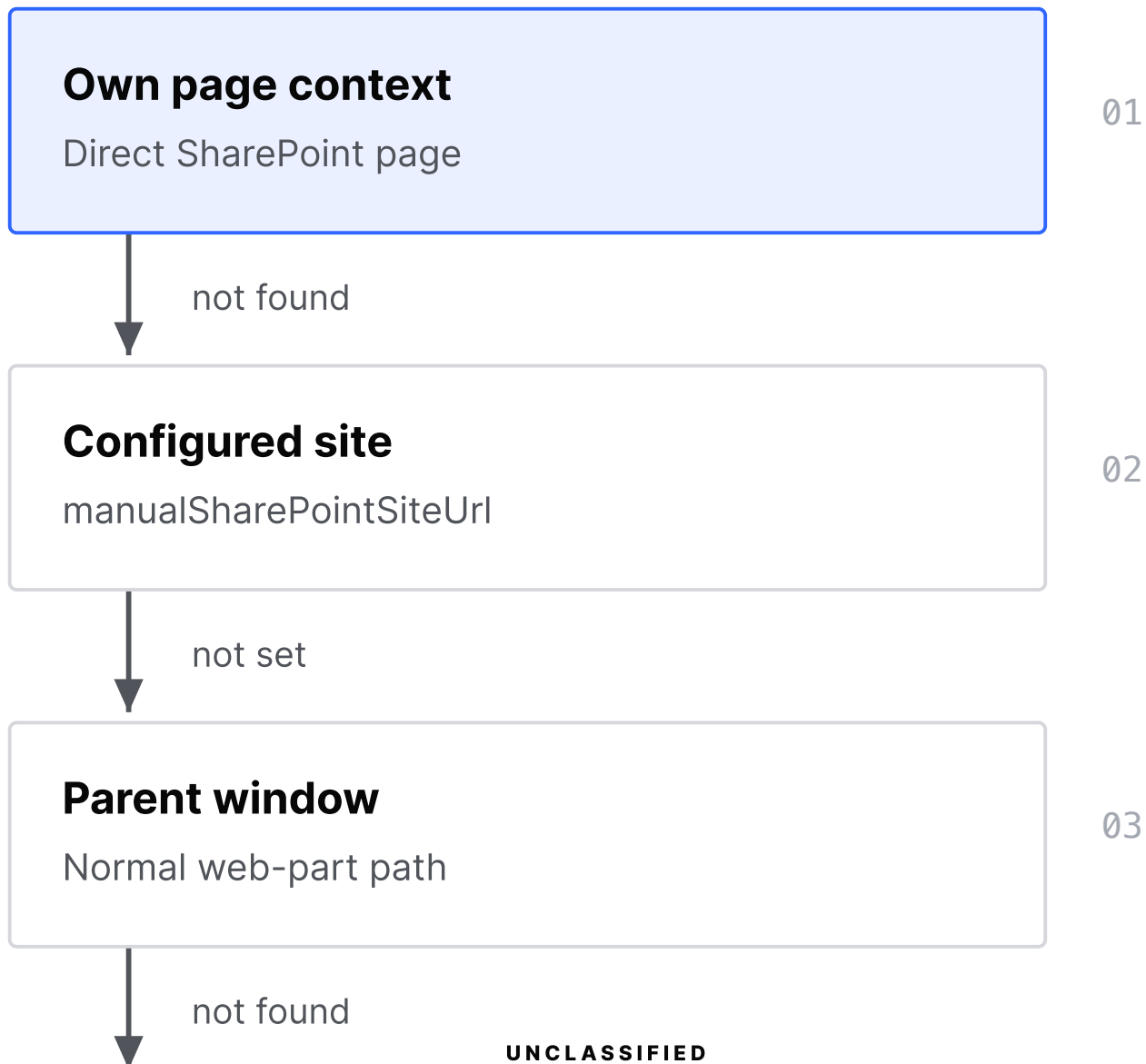
HOSTING

07 / SITE RESOLUTION IN WEB PARTS

A focused package hosted as a SharePoint web part runs inside a sandboxed frame that does not receive the SharePoint page context; only the host page has it. The package therefore resolves the site address through an ordered chain and falls back to local-only behaviour if every step fails.

RESOLUTION ORDER

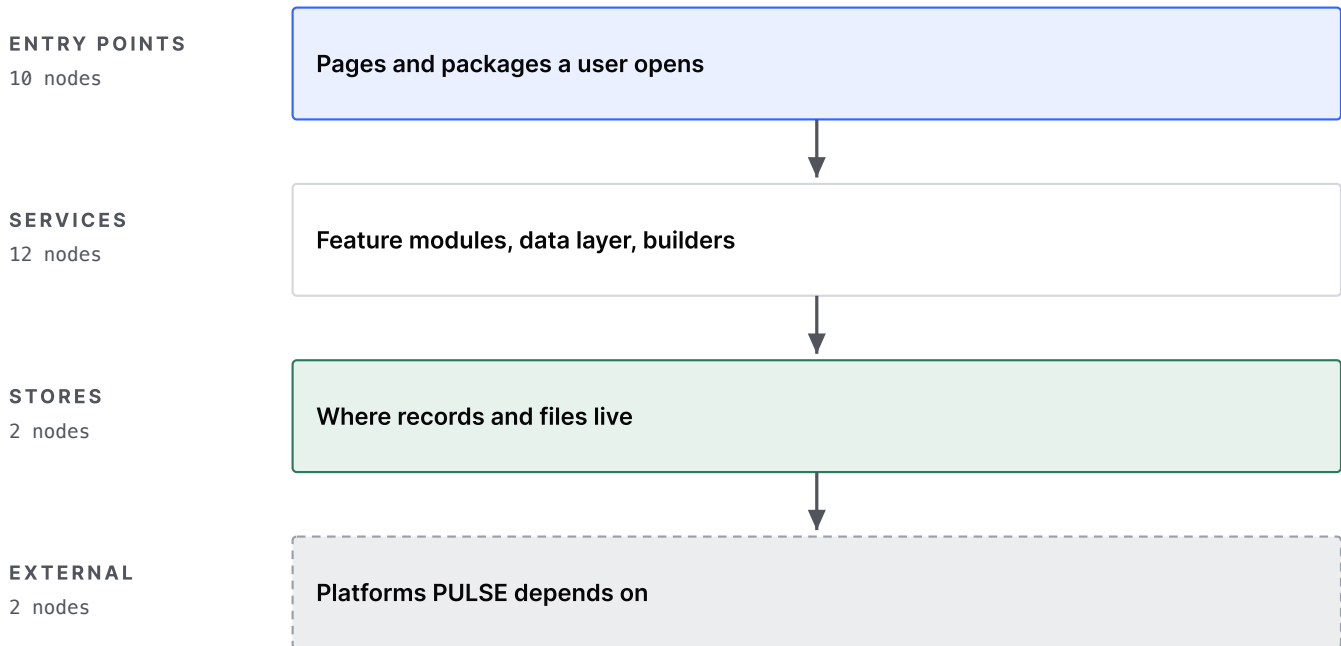
1. The frame's own SharePoint page context, when the package is on a SharePoint page directly.
2. An explicit site address in the application configuration.
3. The parent window's page context, which is the normal path for a web part.
4. The top window's page context, for a nested frame.
5. A server-relative web path combined with the current origin.
6. A site address cached by the full application on a previous successful boot.



SYSTEM MAP

08 / HOW THE CODEBASE FITS TOGETHER

A scan of the source finds 26 components joined by 35 relationships: 10 entry points a user can open, 12 service modules, 2 stores, and 2 external platforms. Every component in the scan carries a source path, and all of them were checked to exist before these figures were published.

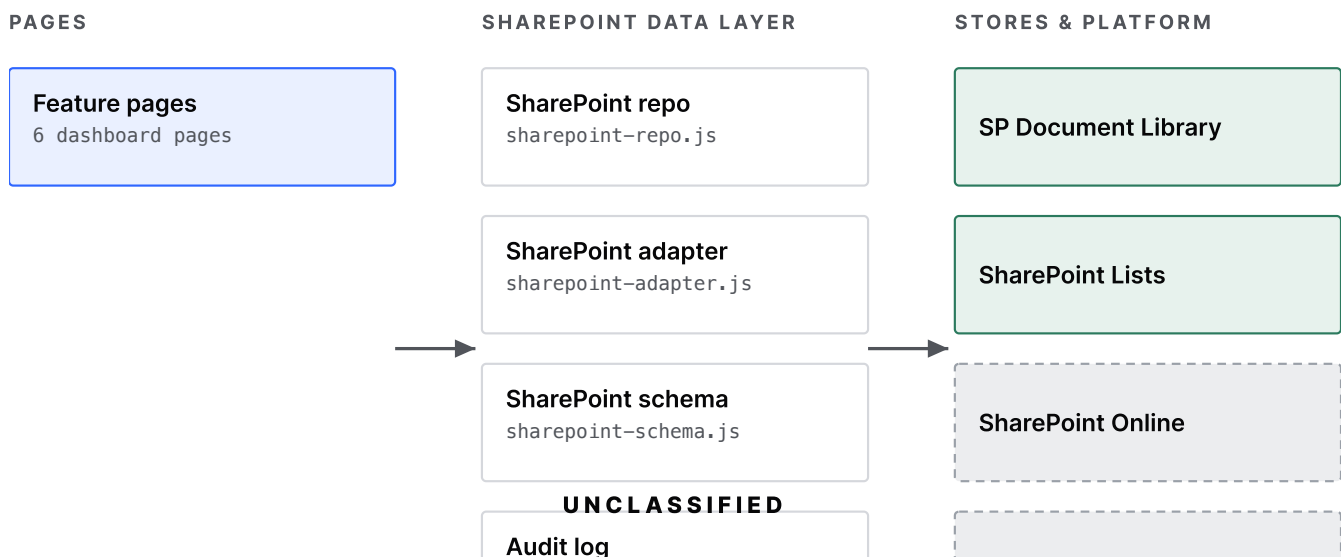


Source: codebase scan of apps/

The layering explains why a change to one shared module reaches every package: the entry points are thin, and almost all behaviour lives in the service tier they share.

THE PATH THAT CARRIES THE RISK

Feature pages never talk to SharePoint directly. They call the SharePoint data layer, which owns the REST transport, the record mapping, the schema, and the audit trail. That is the whole surface where paging ceilings, the view threshold, and permission behaviour apply.



PROCEDURE

09 / PUBLISHING AND VERIFYING A TOOL PAGE

This is the short form of the controlled procedure. The authoritative steps, verification, cautions, and records are in Standard Operating Procedures, SOP 9.

1. Build every package with the unified builder and retain the prior known-good artifact.
2. Upload the package for the intended area and add it to the page through the approved hosted-page process.
3. Confirm site resolution: create a record in the tool and confirm it appears in the full application on the same site after refresh.
4. Confirm navigation is limited to the intended area and no unintended area is reachable.
5. Record which package version is published on which page, so a later regression can be traced to an artifact.

RELEASE READINESS

A tool that has not been shown to write a record visible in the full application has not been verified, however correct it looks on the page.

WHAT VERIFICATION IS NOT

A rendered page proves routing and layout only. It does not prove SharePoint identity, list permissions, notification delivery, or administrator-only control. Those require a controlled transaction on an approved site.